

WindBorne Systems, Firmware Intern

Min-En Wu (Brian Wu) minenbrian@gmail.com UCLA Electrical Engineering, expected June 2028

1. A thing I am proud of

The piece I want to explain is the ESP32 computer vision pipeline I built in the Interconnected and Integrated Bioelectronics Lab at UCLA, from December 2024 to November 2025. It was lab research, not a class project. The cartridge is a ferrobatic liquid handling system. Magnets move droplets through assays, and the controller has to see both the magnet and the droplet while the run is happening.

I put detection on the ESP32 path instead of recording video for later. The stack uses YOLO with SAM2 segmentation, multithreading, and a latency budget that held above 60 frames per second on the edge device. I also trained SAM2 binary masks on about 1,000 images so droplet volume could be read from pixels, and I extended tracking with TubeletGraph, CropFormer, and Mask2Former. The unusual part is that the measurement is the control input. If the frame is late, the magnet move is late, and the assay step is wrong.

A related board level project is Thermloop, a C++ predictive thermal controller I am building now. It treats heat like an electrical network and applies cooling from a causal convolution against a precomputed impulse response, so the decision happens before the temperature limit. The same core targets ESP32-S3, Raspberry Pi, and Arduino.

2. Setup

I write firmware in VS Code with PlatformIO for ESP32 and Arduino targets, and I keep host side Python in the same window. Serial logs sit on a second pane. For analog checks I use LTspice, including a small automation bridge that launches native LTspice from a script so an AC sweep or a transient run does not have to be clicked by hand.

I started in the Arduino IDE, which was enough to blink a pin and too opaque once a project had a pin map, a library version, and a second machine. PlatformIO plus git made the build the same on a lab computer and my laptop. I still use a bare flash tool when the only goal is to see whether a new board enumerates.

3. A board that resets every few hours

I stop changing code first. I want one failing board, one firmware hash, and a log that survives the reset. On ESP32 I read the reset reason on the next boot and write it, with uptime, to a retained register or a short flash record. The useful split is brownout, watchdog, external reset pin, or software restart.

The first tests are electrical. I scope the rail at the module pin, not at the regulator, with a short ground, and I look for a dip when the radio, the camera, or a motor starts. I check the reset pin for a slow rise. I check that a blocking camera or sensor read is not starving the watchdog. I swap in a second board. One board failing points at solder, a crystal, or a connector. Every board failing on a similar interval points at firmware or a load the schematic did not budget.

I change strategy when the reset reason is stable and the rail is clean across an event. Then I disable the radio, then the sensor, then logging, and see which cut moves the interval. A few hour failure is often a counter wrap, a heap leak, or a bus that wedges until the watchdog fires. On the lab camera pipeline the failure mode I watch for is the camera peak lining up with the radio peak. The first experiment is to stagger those windows. The lasting fix is a current budget written next to the pin map.

4. What firmware engineers get wrong

The advice I disagree with is that every driver must be fully abstracted before it is allowed on the board. On a balloon, or on a one off assay rig, the first version should touch the register. A pretty HAL that hides chip errata makes the second bug harder, because the failure is no longer next to the bus transaction that caused it. I would rather have a short driver with the datasheet page in the comment, and extract the interface after two chips have survived it.

I also disagree with treating a blocking wait as a moral failure. It is the right tool while you are proving that a sensor answers. It is the wrong tool once that path shares a core with a control loop or a radio. The mistake is leaving the wait in after the experiment worked. The other common miss is logging only the happy path. The useful byte is the reset reason, the last sensor status, and how full the queue was.

5. A sketchy fix I am still a little proud of

On the droplet pipeline, segmentation was more accurate than the control loop could afford. Waiting for the full mask dropped the frame rate and the magnet move stuttered. For a live assay demo I shipped a coarse bounding box from the detector as the position source, and I ran the heavier mask only on a decimated set of frames for volume. It is sketchy because the box is not the droplet, and a glint can walk the magnet. It kept the run alive at the frame rate the motion needed. The volume path stayed on the mask, so the number we reported was not the number we steered with. I would keep the split. I would not leave the box as the only estimate once the tracker was fast enough.

Portfolio: <https://github.com/bw493>. Role: Firmware Intern. Available for in person work in Redwood City on CPT.